Deploy and verify the smart contracts

Documentation index[1] · ภาษาไทย[2] · Deploy Nuxt[3] · Contract details[4]

Reviewed against the repository on **28 October 2026** (Asia/Bangkok). The supplied deployment workflow supports **Polygon Amoy testnet only, chain ID 80002**. It creates `AmoyTestUSDT` and `ReferralDistributor`. **tUSDT is a valueless test token, not real USDT.** Changing an environment chain ID does not make this workflow suitable for mainnet.

1. Prerequisites and account roles

Run from the repository root on a trusted operator machine with Node.js **24+**, npm, the release source and lockfile, access to an Amoy RPC, and a deployment wallet funded with **test POL**. Contract scripts do not need PostgreSQL. The Nuxt application does need it for accounts and purchase accounting.

```
npm ci --include=dev
test -f .env || cp .env.example .env
```

On Windows PowerShell use `npm.cmd` for npm commands and initialize the file with:

```
if (-not (Test-Path -LiteralPath .env)) { Copy-Item -LiteralPath .env.example -Destination .env }
```

Role / setting	Responsibility
`REFERRAL_OWNER_ADDRESS`	Public initial owner of both contracts; receives the initial token supply and controls owner-only operations. To manage distributor fees in the app, link this external wallet to an admin account
`REFERRAL_FEE_RECIPIENT_ADDRESS`	Public initial platform payout address; may differ from the owner
`REFERRAL_DEPLOYER_PRIVATE_KEY`	Separate deployment-only signing key; its wallet pays test POL gas. Deployment does not automatically make it the owner
`NUXT_REFERRAL_AUTHORITY_PRIVATE_KEY`	Server-only signer for placement, eligibility and completed cycles; its public address must match `quoteSigner()` and needs test POL to relay automatic upgrades
Member wallet	Signs its own approvals/purchases and pays its network fees

Use a dedicated authority key. Keep the owner's signing key in its external wallet. Never put any private key in `NUXT_PUBLIC_*`, commit it, or paste it into a deployment log. Existing environment values take precedence over Node's `.env` loading; avoid stale shell/CI overrides when reviewing or broadcasting.

2. Configure a new deployment

For a **fresh installation**, edit these values in private `.env`:

```
NUXT_EVM_RPC_URL=https://rpc-amoy.polygon.technology/
NUXT_EVM_CHAIN_ID=80002
REFERRAL_OWNER_ADDRESS=
REFERRAL_FEE_RECIPIENT_ADDRESS=
REFERRAL_DEPLOYER_PRIVATE_KEY=
NUXT_REFERRAL_AUTHORITY_PRIVATE_KEY=
NUXT_EVM_TOKEN_ADDRESS=
NUXT_REFERRAL_CONTRACT_ADDRESS=
NUXT_REFERRAL_PAYMENT_MODE=contract
NUXT_REFERRAL_MIN_CONFIRMATIONS=2
```

Fill the owner and recipient with valid nonzero EVM addresses and enter the deployment key securely. Leave the authority blank only for the **first preparation** so the script can generate it. An existing installation must preserve its authority and deployment records. If a token/distributor already exists, use the recovery or replacement sections rather than clearing addresses to bypass duplicate protection.

Prepare the deployment:

```
npm run contracts:prepare
```

This compiles both contracts, fills missing deployment defaults, generates the authority key once if absent, writes it into `.env` without printing it, and saves public addresses/bytecode hashes to `.data/contracts/deployment-plan.json`. No blockchain transaction is sent. It prints the authority's public address; retain that for comparison with `quoteSigner()`.

The compiler comes from the pinned local `solc` package (**0.8.37**); OpenZeppelin Contracts is pinned to **5.6.1**. Compilation uses optimizer **200 runs**, `viaIR: true`, and EVM **Cancun**. Preserve the exact release source, lockfile and artifacts for later verification; compilation uses the project script rather than a `hardhat deploy` command.

3. Review before broadcasting

```
npm run contracts:deploy
```

This recompiles and writes/prints the public plan without broadcasting. Check:

- Network is Amoy/80002, and the RPC actually serves that chain.
- Owner, initial fee recipient and authority are the intended accounts; deployer is funded with test POL.
- The plan's `missing` array is empty and bytecode hashes match the release being deployed.
- This is a new deployment rather than an attempt to recover or replace an existing distributor.

Before deploying a changed contract release, run the relevant local suites:

```
npm run test:distributor
npm run test:verification
```

These tests use local fixtures/simulation. They do not prove the live RPC or explorer is available, and they do not broadcast to public Amoy.

4. Deploy both contracts

After reviewing the concrete plan, broadcast:

```
npm run contracts:deploy -- --broadcast
```

The script checks chain ID 80002, deploys the token followed by the distributor, and waits for **two confirmations per deployment**. This deployment threshold is fixed in the script; the application's referral confirmation setting is separate. Constructor arguments are:

```
AmoyTestUSDT(initialOwner)
ReferralDistributor(tokenAddress, initialOwner, initialFeeRecipient, initialQuoteSigner)
```

After confirmation it writes `NUXT_EVM_TOKEN_ADDRESS`, `NUXT_REFERRAL_CONTRACT_ADDRESS` and `NUXT_REFERRAL_PAYMENT_MODE=contract` into the operator's local `.env`. It does not update a remote application's secrets automatically.

Keep these records in a private release backup:

File	Purpose
`.data/contracts/AmoyTestUSDT.json`, `ReferralDistributor.json`	ABI, bytecode, compiler version and source metadata
`.data/contracts/deployment-plan.json`	Public deployment plan
`.data/contracts/deployment-progress.json`	Original constructor addresses, signed deployment bytes, transaction hashes and deployed addresses
`.env`	Private configuration and matching authority key

Progress is saved **before** broadcasting. If a timeout or partial deployment occurs, keep the same source, constructor values, deployer, configuration and progress file, then rerun the same broadcast command. It resumes the saved transactions. Check chain receipts before changing anything. Avoid using that deployer for unrelated transactions while resuming a pending deployment. Never delete progress simply to retry.

`.data` and `.env` are ignored by Git and need separate backups. Progress contains signed transactions, so treat it as private operational material even though the plan contains only public deployment information.

5. Verify source on the explorer

Set private `ETHERSCAN_API_KEY` with Amoy API access. If blank, the script uses `NUXT_EVM_EXPLORER_API_KEY`. First prepare inputs offline:

```
npm run contracts:verify -- --dry-run
```

The dry run needs the configured addresses and matching build records. It checks source hashes, bytecode and constructors, then writes Standard JSON inputs, encoded constructor arguments and public settings to `.data/contracts/verification/`. It needs no explorer key and makes no network requests.

Submit source verification:

```
npm run contracts:verify
```

This checks the Amoy RPC and deployed code, submits sources through Etherscan V2, skips already verified contracts, and saves request IDs/status to `.data/contracts/verification/status.json`. It sends no blockchain transaction and requires no deployer key. If indexing or verification is still pending, rerun the same command to resume. Both contracts should have verified code at `https://amoy.polygonscan.com/address/CONTRACT_ADDRESS#code`, substituting each deployed address.

The saved deployment record supplies **original** constructor arguments even after owner/fee/signer changes. Without it, verification falls back to the original `REFERRAL_OWNER_ADDRESS`, `REFERRAL_FEE_RECIPIENT_ADDRESS`, and public `REFERRAL_QUOTE_SIGNER_ADDRESS` (or the authority key's derived address). Current on-chain values may differ. Restore the original source and dependencies if hashes differ; recompiling edited source does not fix a mismatch with the deployed contract.

6. Connect Nuxt and check readiness

Transfer these settings to the application's private runtime environment, then restart its Node service:

```
NUXT_EVM_RPC_URL
NUXT_EVM_CHAIN_ID=80002
NUXT_EVM_TOKEN_ADDRESS
NUXT_REFERRAL_CONTRACT_ADDRESS
NUXT_REFERRAL_AUTHORITY_PRIVATE_KEY
NUXT_REFERRAL_PAYMENT_MODE=contract
NUXT_REFERRAL_MIN_CONFIRMATIONS=2
```

Transfer the authority key through your normal private secret-management channel. The application does not need `REFERRAL_DEPLOYER_PRIVATE_KEY` or `ETHERSCAN_API_KEY` for purchases. The separate explorer-history key is optional. Ensure injected hosting variables are updated as well as any env file.

Use the explorer's Read Contract view or an RPC reader to compare:

Contract read	Expected result
Distributor `distributionVersion()`	`3`
Distributor `paymentToken()`	`NUXT_EVM_TOKEN_ADDRESS`
Distributor `owner()`	Intended owner; initially `REFERRAL_OWNER_ADDRESS`
Distributor `feeRecipient()`	Intended recipient; initially `REFERRAL_FEE_RECIPIENT_ADDRESS`
Distributor `quoteSigner()`	Public authority address printed during preparation
Distributor `paused()`	`false` when purchases should be enabled
Token `decimals()` / `symbol()`	`6` / `tUSDT`

Sign in at `/admin/login` before linking the owner wallet. Verify contract status and owner controls in admin settings. Admin access and on-chain ownership are separate permissions; linking a wallet does not transfer ownership.

The token mints **1,000,000 tUSDT** to its initial owner. Each wallet can call `faucet()` once for **400,000 tUSDT**, or receive tokens from the owner; the owner can also mint. The faucet is a contract call, not an npm script or a supplied general member faucet UI. Every sending wallet still needs **test POL** for gas.

Use a test member wallet on Amoy to buy level 1 for **10 tUSDT**. The confirmation offers an exact-price approval or a finite approval for this and the remaining levels to enable automatic upgrades. At level 1 that ceiling is **327,670 tUSDT**; only the current price is charged now. Sign the purchase, wait for confirmations, then check activation, hash and accounting history. The server does not precheck purchase POL balances. Contract-mode payments do not fall back to internal ledger credit.

Version 3 pays matrix ancestors **10% / 20% / 50%** at steps 1–3, continuous binary unilevel ancestors **1% each for fifteen steps starting at the parent**, and the platform **5%**. Missing or locked recipients' shares go to the platform. Recipients must own the purchased level. Matrix and unilevel recipients can differ. See current payment rules[5] and ภาษาไทย[6].

A full paid matrix has **2 / 4 / 8 descendants**, three steps below its owner. Completion opens another same-level slot and queues a full-price next-level purchase from the member's wallet. The worker checks every fifteen seconds. Insufficient token balance or approval leaves the upgrade queued. Fund the public `quoteSigner()` wallet with test POL for relayer gas. The authority attests cycle completion; the contract checks the signature, sequential level and one-time cycle hash before spending approved tokens.

7. Replace only the distributor

The contracts are **not upgradeable**. Editing Solidity does not update deployed code. The replacement command reuses the configured token and its balances while deploying a new distributor; it does not migrate on-chain transaction history.

Finish pending purchases, approvals, transfers and fee updates before changing configuration. Schedule a maintenance window and prevent new application writes during the switch. Back up PostgreSQL, private environment and the **original** `.data/contracts` before running preparation/compilation against changed source, because those commands overwrite build artifacts. Keep the reused token's original build records too.

With the current token and distributor still configured, review:

Both public address fields must be filled for replacement. For the installation recorded on 28 October 2026, the existing values were:

```
NUXT_EVM_TOKEN_ADDRESS=0x8B8B3C4115d1F520688F5d0537b6854eb4258065
NUXT_REFERRAL_CONTRACT_ADDRESS=0x485cC2Cd52fBC7adeEAdf175b37f9c50741FdCB6
NUXT_REFERRAL_PAYMENT_MODE=contract
```

These are public addresses, not private keys. Confirm them against your current installation before using them; after a successful replacement, keep the newly saved distributor address. Blank addresses do not mean that the contracts have disappeared from the chain. In PowerShell use `npm.cmd` if execution policy blocks `npm`.

Prepare the replacement without sending a transaction:

```
npm run contracts:deploy -- --replace-distributor
```

Review `.data/contracts/replacement-plan.json`. Replace `CURRENT_DISTRIBUTOR_ADDRESS` below with the exact current distributor address, then broadcast:

```
npm run contracts:deploy -- --replace-distributor --previous-contract CURRENT_DISTRIBUTOR_ADDRESS --broadcast
```

For the recorded distributor above, the complete PowerShell command is:

```
npm.cmd run contracts:deploy -- --replace-distributor --previous-contract
0x485cC2Cd52fBC7adeEAdf175b37f9c50741FdCB6 --broadcast
```

The reviewed plan must show `distributionVersion: 3`, `broadcast: false`, the intended owner/fee/authority, the existing token, the exact old distributor in `replaces`, and `missing: []`. Adding `--broadcast` sends the deployment and spends the deployer's test POL. Wait for `Deployment confirmed` and the saved new address. If interrupted, resume the same command using its saved progress.

The script checks the old distributor's token, deploys only the new distributor, preserves the original deployment progress, and saves replacement records to `.data/contracts/replacement-ADDRESS.json` and `.data/contracts/latest-replacement.json`. The local `.env` is updated to the new distributor; publish that value to the Nuxt runtime environment and restart.

Repeat source verification and readiness checks. Existing database accounts and paid-level accounting are retained; no database reset is required. Previous purchases remain transactions of the old contract. Never clear the configured address to force a fresh two-contract deployment when the intention is replacement.

After the confirmed replacement, run:

```
npm.cmd run db:migrate
npm.cmd run contracts:verify -- --dry-run
npm.cmd run contracts:verify
npm.cmd run build
```

Restart the application with the new runtime configuration. Migration `0013_matrix_slots_and_auto_upgrades.sql` preserves paid levels, adds matrix slots and durable upgrade jobs, and avoids retroactive automatic charges for pre-existing full matrices. Members must approve the **new distributor** before it can spend their tokens; old allowances do not transfer. Recheck `distributionVersion() == 3`, token, signer, pause state and relayer gas before reopening purchases.

8. Recovery and configuration changes

Problem	Action
Wrong chain / RPC failure	Use an Amoy RPC and chain ID 80002; retain the saved progress while investigating
Gas insufficient / deployment timeout	Fund the same deployer with test POL, inspect saved hashes/receipts, then resume the same command
Duplicate deployment rejected	Inspect the existing configuration and progress; use replacement only when intentional
Quote signer mismatch	Restore the authority matching `quoteSigner()`; creating a new `.env` key does not update the contract
Fee/owner values changed only in `.env`	These are constructor inputs. Existing contract changes require the current owner's signed transaction; ownership transfer is two-step
Source verification mismatch	Restore the deployed source, lockfile, compiler and original constructor records
Purchase still pending	Keep the original ID/hash and allow receipt reconciliation; do not change contract/payment mode mid-operation

If rotating the quote signer, coordinate the owner's on-chain update and the application's authority key, then restart and recheck. Fee/signer updates invalidate outstanding quotes. A database restore or an application rollback cannot reverse blockchain transfers or contract deployment.

Source references: deployment[7], preparation[8], compiler[9], verification[10], distributor[11], test token[12].

Repository references

Numbered references identify repository files listed below. Include the source repository when sharing documents that require those files. External website links remain clickable in the PDF.

1. `docs/README.md`
2. `docs/th/deploy-contracts.md`
3. `docs/en/deploy-nuxt.md`
4. `docs/referral-contract.md`
5. `docs/matrix-payments.md`
6. `docs/th/matrix-payments.md`
7. `scripts/deploy-referral.mjs`
8. `scripts/lib/referral-deployment.mjs`
9. `scripts/lib/compile-contracts.mjs`
10. `scripts/verify-referral.mjs`
11. `contracts/ReferralDistributor.sol`
12. `contracts/AmoyTestUSDT.sol`